

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №2
з курсу «Системи штучного інтелекту»
на тему:
**«Сліпий пошук на графах. Особливості реалізації пошуку в глибину у
графах»**

Виконав:
студент групи ФсІ-42
Трофимов Володимир
Перевірів:
ас. Іжик О. Б.

Зміст

Тема	3
Мета	3
Вхідні дані	3
Завдання	3
Критерії виконання роботи	3
Теоретичні відомості	4
1. Графи та їх основні поняття	4
2. Основні види графів	5
2.1. Неорієнтований граф	5
2.2. Орієнтований граф	5
3. Дерево.....	6
4. Пошук у графах	6
5. Пошук у глибину	7
6. Стек у DFS	7
7. Алгоритм DFS	7
8. Відновлення знайденого шляху	8
9. DFS та найкоротший шлях	9
10. Повернення назад у DFS	9
11. Порівняння BFS та DFS	10
Інтерфейс та принцип роботи програми	11
Результати дослідження роботи програми	15
<i>Дослідження 1: зміна напрямку обходу вершин на неорієнтованому графі.</i>	15
<i>Дослідження 2: зміна початку та кінця пошуку місцями.</i>	17
<i>Дослідження 3: Видалення та додавання вершин, ребер та дуг, які входять у вирішальний граф.</i> ..	17
<i>Дослідження 4: Використання пошуку вглиб на різних видах графів.</i>	19
<i>Дослідження 5: Порівняння пошуку вищир та пошуку вглиб</i>	22
Висновок	27
Код програми	29

Тема

Сліпий пошук на графах. Особливості реалізації пошуку в глибину у графах.

Мета

Створити програму, яка реалізує пошук в ширину на графі, та дослідити особливості її роботи на графах різних видів та при різних умовах пошуку. Порівняти особливості пошуків в ширину та глибину на графах.

Вхідні дані

- звичайний розгалужений граф (не менше 5-и гілок) порядку не менше 30 та розміром не менше 30-40 (не менше 5-и “гілок” у випадку дерева; граф повинен бути заданий явно, з зафіксованим положенням (координатами) вершин; рандомне задання графа не допускається)
- початкова і цільова вершини графу;
- порядок обходу вершин графу при пошуку.

Завдання

Створити програму виконання пошуку в глибину на графі, використовуючи одну з мов програмування високого рівня, в якій реалізувати:

- візуально представити заданий граф;
- знаходження найкоротшого шляху від вказаної початкової до цільової вершини, використовуючи алгоритм пошуку в глибину;
- виведення результатів пошуку;
- відмітити переваги і недоліки цього виду пошуку.

Критерії виконання роботи

1. Створити графічний інтерфейс керування роботою програми, який має передбачати можливість проводити з його використанням всі потрібні для виконання завдання операції, у т. ч.:

- візуалізацію графу, самого процесу пошуку і представлення його результатів та часу виконання пошуку;
- можливість змінювати розмірність (введення і видалення вершин) і порядок (додавання і видалення дуг і ребер) графу, а також виду графу (звичайного, оргграфу та дерева);
- можливість змінювати напрям обходу вершин графу при пошуку; – представлення результатів пошуку (знайдений шлях на графі при заданих вершинах, кількість використаних при його знаходженні циклів та кількість розкритих вершин), які виводяться в окремому вікні (вікнах) інтерфейсу.

2. Дослідити особливості роботи написаної програми та результати пошуку, отримані при:

- різних напрямках обходу суміжних вершин при розкритті вершини;
- зміні напрямку пошуку (при дзеркальній заміні початкової вершини у і мети пошуку);
- видалення та додавання вершин, ребер та дуг, які входять у вирішальний граф;
- вплив на результати пошуку виду графа – дослідити особливості пошуку вшир на дереві, звичайному та орієнтованому графах однакового порядку та розміру і різних розмірів, а також заміни деяких ребер графа на однонаправлені дуги.

3. Порівняти результати роботи алгоритмів пошуку вшир та вглиб за різних умов пошуку, використавши в кожному випадку для цього один і той самий граф та однакові умови пошуку. Пояснити, в чому полягає основна відмінність організації пошуку на графах в ширину та глибину.

Теоретичні відомості

1. Графи та їх основні поняття

Граф — математична структура, що складається з множини вершин та множини зв'язків між ними. Граф позначається:

$$G = (V, E),$$

де:

- V — множина вершин графа;
- E — множина ребер або дуг, що з'єднують вершини.

Вершина є елементом графа, який може представляти деякий об'єкт або стан системи. *Ребро* задає зв'язок між двома вершинами.

Наприклад:

$$V = \{v_1, v_2, v_3, v_4\},$$
$$E = \{(v_1, v_2), (v_1, v_3), (v_2, v_4)\}.$$

Граф може задаватися за допомогою списків суміжності, матриці суміжності або графічно.

Порядок та розмір графа

Порядком графа називається кількість його вершин:

$$|V| = n.$$

Розміром графа називається кількість його ребер або дуг:

$$|E| = m.$$

Таким чином, для графа з 30 вершинами та 40 ребрами:

$$n = 30, m = 40.$$

2. Основні види графів

2.1. Неорієнтований граф

У неорієнтованому графі ребро не має напрямку.

Якщо існує ребро:

$$(v_1, v_2),$$

то з вершини v_1 можна перейти до v_2 , а з v_2 — до v_1 .

Графічно таке ребро зображається звичайною лінією:

$$v_1 - v_2$$

Для DFS це означає, що при розкритті вершини необхідно враховувати всі вершини, з'єднані з нею ребрами.

2.2. Орієнтований граф

Орієнтований граф, або *орграф*, — це граф, у якому кожен зв'язок має певний напрямок.

Зв'язок між вершинами називається *дугою*:

$$v_1 \rightarrow v_2.$$

У цьому випадку перехід з v_1 до v_2 дозволений, а з v_2 до v_1 — не обов'язково.

Наприклад:

$$v_1 \rightarrow v_2 \rightarrow v_3.$$

Під час BFS напрямок дуг необхідно враховувати. Якщо існує лише:

$$v_1 \rightarrow v_2,$$

то під час пошуку від v_2 до v_1 цей зв'язок використовуватися не може.

Саме тому заміна частини ребер неорієнтованого графа на односпрямовані дуги може призвести до:

- зміни знайденого шляху;

- збільшення довжини шляху;
- відсутності шляху;
- зміни кількості розкритих вершин.

3. Дерево

Дерево — це зв'язний неорієнтований граф, у якому між будь-якими двома вершинами існує рівно один простий шлях.

Для дерева з n вершинами кількість ребер завжди дорівнює:

$$m = n - 1.$$

Наприклад, для 30 вершин дерево має:

$$m = 29.$$

Основними властивостями дерева є:

- відсутність циклів;
- зв'язність;
- між двома будь-якими вершинами існує єдиний шлях;
- додавання нового ребра між уже існуючими вершинами створює цикл.

Для пошуку в ширину дерево є особливо зручним випадком, оскільки між початковою та цільовою вершинами існує тільки один шлях. Тому зміна порядку обходу суміжних вершин не змінює сам шлях, хоча може змінювати порядок розкриття вершин.

4. Пошук у графах

Пошук у графі — це процес систематичного переходу від одних вершин графа до інших для знаходження вершини, яка задовольняє задану умову.

Зазвичай задаються:

- початкова вершина S ;
- цільова вершина G ;
- граф $G = (V, E)$;
- порядок розгляду суміжних вершин.

У лабораторній роботі необхідно знайти шлях:

$$S \rightarrow \dots \rightarrow G.$$

Існують різні алгоритми пошуку, зокрема:

- пошук у ширину (*Breadth-First Search, BFS*);
- пошук у глибину (*Depth-First Search, DFS*);
- пошук з обмеженням глибини;

- пошук з ітеративним поглибленням;
- евристичний пошук тощо.

У цій лабораторній роботі використовується пошук у ширину.

5. Пошук у глибину

Пошук у глибину (Depth-First Search, DFS) — алгоритм обходу графа, який досліджує один із напрямків якомога глибше, перш ніж повернутися назад і продовжити пошук в іншому напрямку.

Принцип роботи можна представити так:

- початкова вершина
- вершина першого рівня
- вершина другого рівня
- вершина третього рівня
- ...
- повернення назад
- наступна гілка

Тобто DFS спочатку намагається пройти якомога далі по одній гілці, і лише коли подальший рух неможливий, повертається до попередніх вершин та досліджує інші напрямки. На відміну від BFS, який обходить граф пошарово, DFS рухається переважно вглиб графа.

6. Стек у DFS

Основною структурою даних алгоритму DFS є стек (Stack).

Стек працює за принципом LIFO (Last In, First Out), тобто останній доданий елемент буде першим вилучений.

Наприклад:

- Додали: $A \rightarrow B \rightarrow C \rightarrow D$
- Стек: $[A, B, C, D]$
- Вилучаємо D: $[A, B, C]$
- Вилучаємо C: $[A, B]$

Саме використання LIFO забезпечує можливість DFS спочатку продовжувати дослідження останньої доданої вершини, тобто рухатися вглиб поточної гілки.

У програмній реалізації DFS стек може використовуватися явно, наприклад через структуру stack, або неявно — за допомогою рекурсії.

7. Алгоритм DFS

Нехай задано:

- S — початкова вершина;
- G — цільова вершина.

Основний алгоритм можна описати так:

1. Створити порожній стек.
2. Додати початкову вершину S до стеку.
3. Позначити S як відвідану.
4. Поки стек не порожній:
 - вилучити вершину з вершини стеку;
 - перевірити, чи є вона цільовою;
 - переглянути її суміжні вершини;
 - невідвідані вершини додати до стеку;
 - позначити їх як відвідані.
5. Якщо цільову вершину знайдено — відновити шлях.
6. Якщо стек спорожнів, а цільову вершину не знайдено — шляху не існує.

Важливо враховувати порядок додавання суміжних вершин до стеку. Оскільки стек працює за принципом LIFO, остання додана вершина буде досліджена першою.

Наприклад, якщо для вершини A доступні вершини:

$$A \rightarrow B, C, D$$

і вони додаються до стеку саме в такому порядку:

$$\text{Стек: } [B, C, D]$$

то першою буде вилучена:

$$D$$

Тому порядок перегляду суміжних вершин безпосередньо впливає на результат обходу DFS.

8. Відновлення знайденого шляху

Як і у BFS, для відновлення повного шляху в DFS необхідно запам'ятовувати попередника кожної відвіданої вершини.

Наприклад, під час пошуку було отримано:

$$A \rightarrow C \rightarrow F \rightarrow G$$

Тоді:

$$\text{parent}[C] = A$$

$$\text{parent}[F] = C$$

$$\text{parent}[G] = F$$

Після знаходження цільової вершини G шлях відновлюється у зворотному напрямку:

$$G \leftarrow F \leftarrow C \leftarrow A$$

Після розвороту отримуємо:

$$A \rightarrow C \rightarrow F \rightarrow G$$

Таким чином, масив `parent` дозволяє відновити послідовність вершин від цільової до початкової.

9. DFS та найкоротший шлях

На відміну від BFS, звичайний DFS не гарантує знаходження найкоротшого шляху у неваговому графі.

Наприклад, нехай існують два шляхи:

$$A \rightarrow B \rightarrow G$$

та

$$A \rightarrow C \rightarrow D \rightarrow E \rightarrow G$$

Якщо DFS спочатку вибере напрямок через C , він може знайти:

$$A \rightarrow C \rightarrow D \rightarrow E \rightarrow G$$

Довжина цього шляху становить 4 ребра.

Однак існує коротший шлях:

$$A \rightarrow B \rightarrow G$$

його довжина становить лише 2 ребра.

Тому DFS може знайти перший доступний шлях, але цей шлях не обов'язково буде найкоротшим.

Отже:

$$L = \text{кількість ребер у знайденому шляху}$$

Для DFS значення L не обов'язково є мінімальним серед усіх можливих шляхів між початковою та цільовою вершинами.

Важливо: якщо необхідно гарантовано знайти найкоротший шлях у неваговому графі, для цього доцільно використовувати BFS, а не звичайний DFS.

10. Повернення назад у DFS

Характерною особливістю DFS є механізм повернення назад (backtracking).

Якщо з поточної вершини немає невідвіданих суміжних вершин, алгоритм повертається до попередньої вершини та перевіряє, чи існує там інший невідвіданий напрямок.

Наприклад:

$$A \rightarrow B \rightarrow D \rightarrow G$$

Якщо з вершини G більше немає доступних невідвіданих вершин, DFS повертається:

$$G \leftarrow D$$

Якщо з D також немає інших напрямків:

$$D \leftarrow B$$

і продовжує перевірку інших гілок.

Саме цей механізм дозволяє DFS систематично досліджувати всі доступні частини графа.

11.Порівняння BFS та DFS

Характеристика	BFS	DFS
Принцип	пошук у ширину	пошук у глибину
Основна структура	черга	стек
Принцип роботи	FIFO	LIFO
Напрямок пошуку	пошарово	вглиб
Повернення назад	не є основним механізмом	характерне для алгоритму
Найкоротший шлях у неваговому графі	гарантує	не гарантує
Робота з циклами	потрібні відвідані вершини	потрібні відвідані вершини
Залежність від порядку сусідів	є	особливо суттєва

Таким чином, BFS доцільно використовувати, коли важливо знайти найкоротший шлях за кількістю ребер, а DFS — коли необхідно дослідити структуру графа, пройти його гілки вглиб або виконати задачі, пов'язані з пошуком, рекурсією та backtracking.

Інтерфейс та принцип роботи програми

Графічний інтерфейс розробленої програми реалізовано мовою Python з використанням бібліотеки CustomTkinter, яка забезпечує створення сучасних елементів графічного інтерфейсу та їх взаємодію з алгоритмічною частиною програми. Структуру інтерфейсу організовано у вигляді трьох основних функціональних областей: панелі навігації між вкладками, панелі керування параметрами графа та алгоритму BFS і області візуалізації графа та процесу пошуку.

Основна функціональність програми зосереджена у вкладці «Лабораторна», яка призначена для виконання експериментів та дослідження особливостей алгоритму пошуку в ширину. Додатково передбачено вкладки «Результати» та «Теорія BFS та DFS» (див. рис.1).

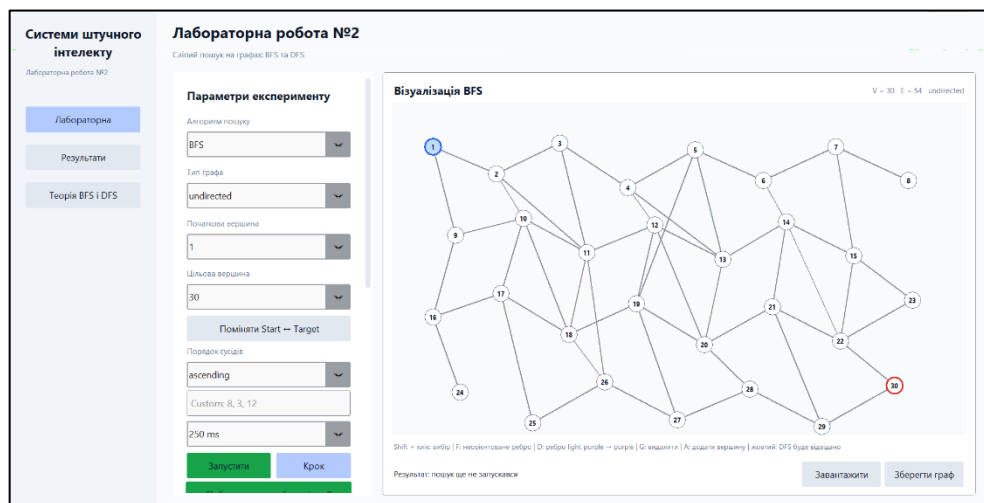


Рис.1 – Загальний вигляд інтерфейсу.

Вкладка «Лабораторна» є основним робочим середовищем програми та містить засоби налаштування параметрів експерименту, керування графом, запуску алгоритму пошуку та його візуалізації.

У секції «Параметри експерименту» користувач може вибрати алгоритм, задавати основні параметри пошуку та структури графа. Передбачено можливість вибору типу графа: неорієнтований граф, орієнтований граф або дерево. Це дозволяє досліджувати вплив структури та напрямленості зв'язків на результати роботи алгоритму BFS та DFS.

Також задаються початкова (Start) та цільова (Target) вершини пошуку. Їх можна задати у полях “Початкова вершина” та “Цільова вершина” або натиснути на графі лівою кнопкою миші на початкову вершину, а якщо натиснути лівою кнопкою миші та ctrl – цільову вершину. Для проведення експериментів зі зміною напрямку пошуку передбачено функцію «Поміняти Start ↔ Target», яка автоматично змінює початкову та цільову вершини місцями.

Окремим параметром є порядок обходу суміжних вершин. Програма підтримує декілька режимів: обхід у зростаючому порядку, у спадаючому порядку та за спеціально заданою користувачем послідовністю. Даний параметр використовується для дослідження впливу порядку розгляду суміжних вершин на процес BFS/DFS та вибір конкретного шляху у випадку існування декількох найкоротших маршрутів.

Для керування швидкістю демонстрації алгоритму передбачено параметр часу виконання одного кроку. Він використовується під час покрокової анімації пошуку та дозволяє спостерігати послідовність розкриття вершин і зміни стану графа.

Основні операції керування пошуком представлені такими функціями (див. рис.2):

- «Запустити» — виконує пошук у ширину/глибину (залежить відповідно від вибраного алгоритму: BFS/DFS) від заданої початкової вершини до цільової;
- «Крок» — виконує один етап алгоритму, що дає змогу детально простежити послідовність роботи BFS/DFS;
- «Побудувати шлях без анімації» — виконує пошук та відображає кінцевий результат без покрокової візуалізації;
- «Очистити пошук» — видаляє результати поточного пошуку та повертає граф до початкового стану.

The image shows a vertical control panel for a search algorithm experiment. It contains several dropdown menus and buttons. The first dropdown is labeled 'Алгоритм пошуку' (Search algorithm) and is set to 'DFS'. The second is 'Тип графа' (Graph type) set to 'undirected'. The third is 'Початкова вершина' (Start vertex) set to '1'. The fourth is 'Цільова вершина' (Target vertex) set to '30'. Below these is a button 'Поміняти Start ↔ Target'. The next dropdown is 'Порядок сусідів' (Neighbors order) set to 'ascending', with a text input below it showing 'Custom: 8, 3, 12'. Below that is a dropdown for time delay set to '250 ms'. At the bottom are four buttons: 'Запустити' (Run) in green, 'Крок' (Step) in blue, 'Побудувати шлях без анімації' (Build path without animation) in green, and 'Очистити пошук' (Clear search) in light blue.

Рис.2 –Вигляд (основної частини) параметрів експерименту.

Для дослідження різних конфігурацій графа реалізовано секцію «Редагування графа». За її допомогою можна змінювати структуру графа шляхом додавання та видалення вершин, ребер і дуг. Окрім полів вводу реалізовано вибір вершини/вершин за допомогою натискання клавіші shift та лівому кліку миші. Якщо вибрано одну вершину, то її можна видалити, вибравши її та натиснувши на клавішу “G”. Якщо необхідно додати вершину, можна задати її номер та натиснути на “Додати вершину” або натиснути на клавішу “A” (в такому випадку додається вершина, для якої не зайнятий id починаючи від 1). Ці вершини також можна переміщувати: натиснути лівою кнопкою миші на вершину та не відпускати для переміщення. Реалізовано також додавання/видалення ребер/дуг, вибравши відповідні дві вершини та натиснувши на такі клавіші:

- D – додати дугу від першої вершини (позначеною світло-фіолетовим кольором) до другої вершини (позначеною фіолетовим кольором);
- F – додати ребро між двома вершинами;
- G – видалити дугу/ребро.

Такий функціонал дозволяє досліджувати вплив зміни порядку та розміру графа на результати пошуку і легко редагувати граф.

Крім того, передбачено можливість генерування випадкового графа. Дана функція має допоміжний характер і використовується для додаткового тестування працездатності програмної реалізації. Основний граф, використаний для виконання лабораторного дослідження, задається явно із заздалегідь визначеними вершинами, ребрами та координатами їх розташування, що відповідає вимогам лабораторної роботи.

Для швидкого повернення до початкової конфігурації передбачено функцію «Reset Graph». Вона відновлює вихідну структуру графа після виконання експериментів з додаванням або видаленням його елементів.

Центральну частину робочого вікна займає область візуалізації графа. Вершини графа відображаються у фіксованих координатах, а зв'язки між ними представлені відповідними ребрами або напрямленими дугами. Для орієнтованих графів напрямок переходу додатково позначається стрілками.

Під час виконання BFS/DFS візуально відображаються проміжні стани алгоритму (див. рис.3). Це дозволяє простежити порядок розкриття вершин, формування черги та поступове наближення до цільової вершини. Зміна візуального стану вершин використовується для розмежування невідвіданих, відвіданих, поточної та розкритої вершин. Після успішного завершення пошуку знайдений маршрут додатково виділяється на графі.

Такий підхід забезпечує наочне представлення алгоритму та дозволяє безпосередньо співвіднести теоретичний принцип пошуку в ширину з фактичною послідовністю обробки вершин.

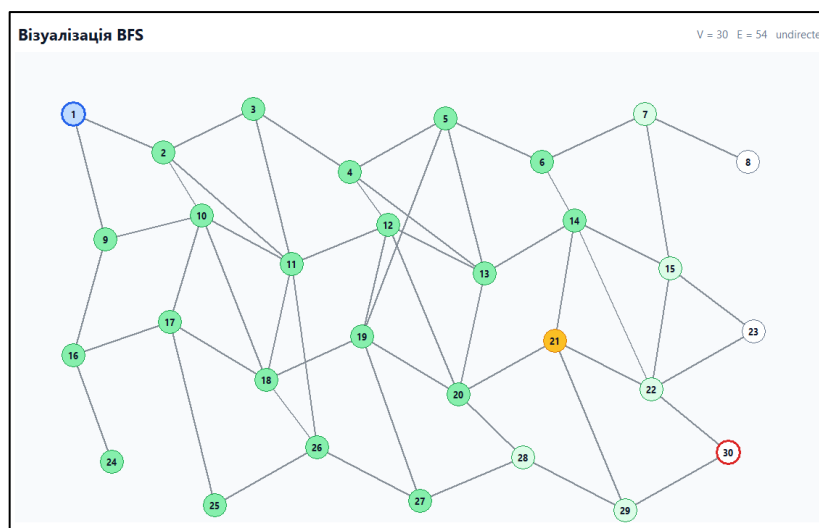


Рис.3 – Візуалізація графа

Вкладка «Результати» призначена для фіксації результатів проведених експериментів. Результати можуть зберігатися у вигляді окремих записів, що дозволяє порівнювати різні конфігурації графа та параметри пошуку.

Для кожного експерименту можуть фіксуватися основні характеристики виконання BFS, зокрема початкова та цільова вершини, тип графа, порядок обходу суміжних вершин, знайдений шлях, його довжина, кількість розкритих вершин, кількість циклічних переходів та час виконання алгоритму.

Додавання запису до таблиці результатів здійснюється користувачем після завершення відповідного експерименту (див. рис.4). Це дозволяє зберігати лише ті результати, які безпосередньо використовуються під час подальшого аналізу.

Результати та експерименти											
Зберігайте результати запусків для порівняння типів графів і порядку обходу.											
Додати поточний результат Очистити таблицю											
№	Type	V	E	Start	Target	Order	Path	Expanded	Iterations	Time	
1	undirected	30	54	1	30	ascending	7	30	30	7394.031 ms	

Рис.4 – Вкладка «Результати»

Вкладка «Теорія BFS і DFS» (див. рис.5) містить стислий довідковий матеріал щодо алгоритму пошуку в ширину. У ній наведено основний принцип роботи BFS/DFS, особливості використання черги, порядок розкриття вершин, умови знаходження найкоротшого шляху та основні характеристики алгоритму.

Наявність теоретичного розділу безпосередньо в програмному середовищі забезпечує можливість оперативного звернення до необхідних теоретичних відомостей під час проведення експериментів.

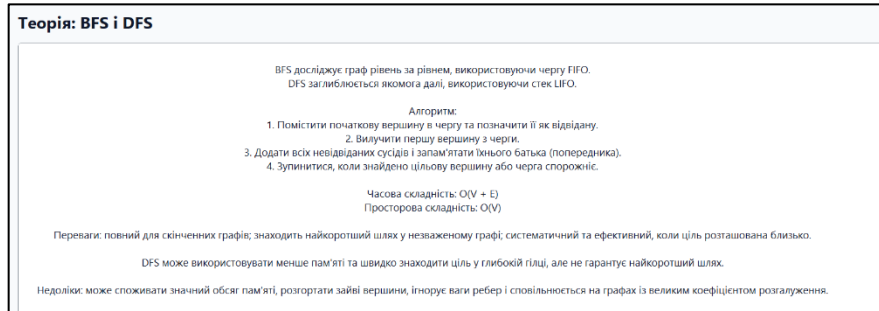


Рис.5 – Вкладка «Теорія BFS і DFS»

Таким чином, розроблений графічний інтерфейс об'єднує засоби формування та редагування графа, налаштування параметрів пошуку, покрокового виконання BFS/DFS, візуального спостереження за його роботою та фіксації отриманих результатів. Це забезпечує необхідну функціональність для проведення досліджень впливу структури графа, його розміру, порядку обходу суміжних вершин та напрямку пошуку на результати роботи алгоритму пошуку в ширину/глибину.

Результати дослідження роботи програми

Дослідження 1: зміна напрямку обходу вершин на неорієнтованому графі.

Перший етапом буде обхід графа за зростанням. Для цього наперед було задано неорієнтований граф із 30 вершин та 54 ребер.

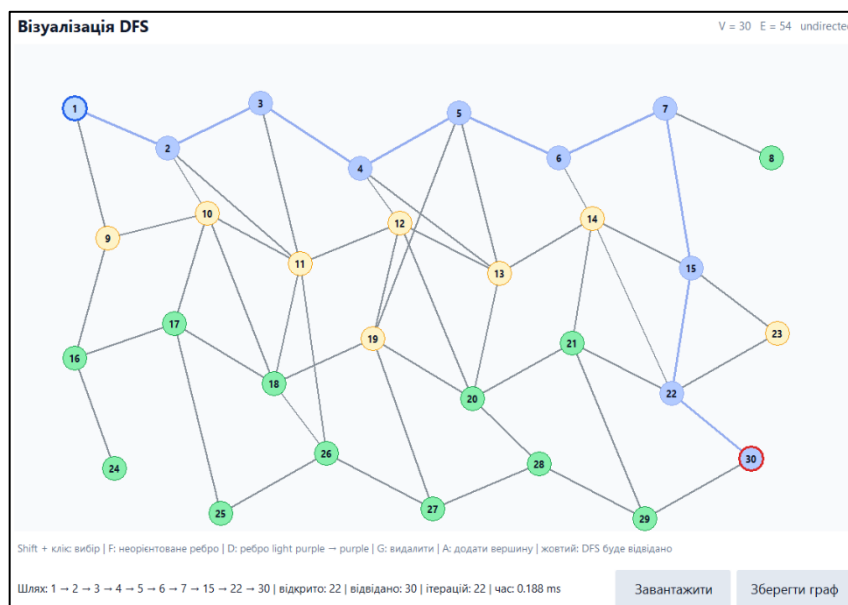


Рис.6 – Обхід неорієнтованого графа за зростанням

Згідно рисунка 6, даний обхід показав такі результати: кількість відкритих вершин 22, відвіданих: 30, що вказує на повний обхід графа для побудови найкоротшого маршруту. При цьому був витрачений 0.188 с.

Тепер використаємо обхід за спаданням та власний обхід на тому ж самому графі (цей обхід використовує обхід за зростанням та додатково наші правила обходу). Для власного обходу створено правило: 30 має більший пріоритет ніж 21.

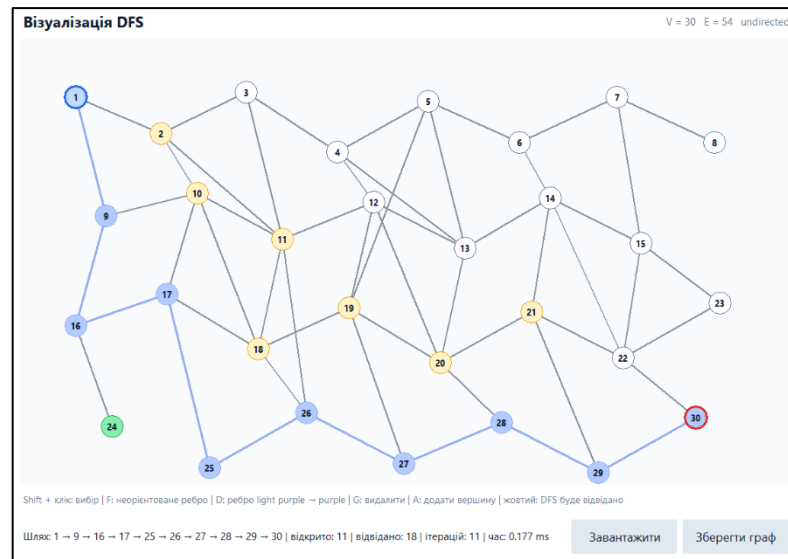


Рис.7 – Обхід неорієнтованого графа за спаданням

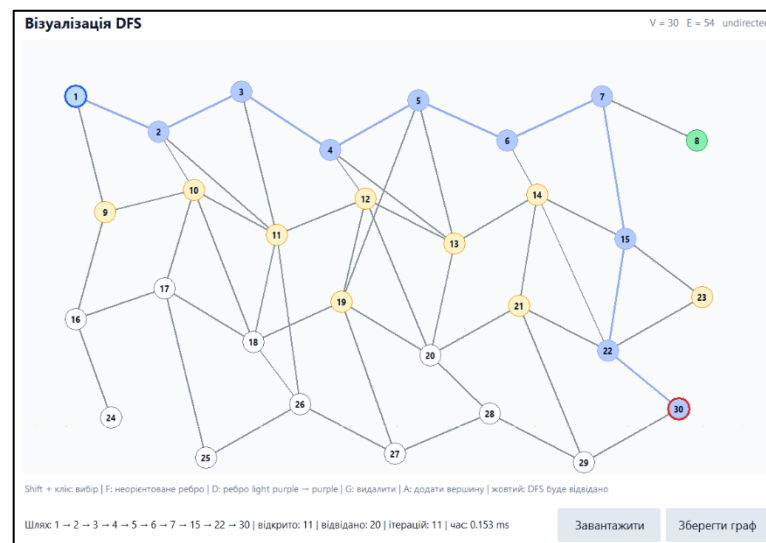


Рис.8 – Обхід неорієнтованого графа за власним порядком (в даному випадку створено правило: 30 має більший пріоритет ніж 21).

Як видно на рисунку 7 та рисунку 8 результати змінилися для даного графу. Обхід за спаданням показав час 0.177 с., кількість відкритих вершин 11, відвіданих - 18, а за власним порядком (30 має більший пріоритет ніж 21) 0.153 с., кількість відкритих вершин 11, відвіданих - 20. В цьому експерименті для даного графу найкраще показав себе обхід за спаданням (показник: кількість відкритих вершин) або за власним обходом (показник час, однак чи можна вважати це як результат, який варто брати до уваги – не впевнений).

Дослідження 2: зміна початку та кінця пошуку місцями.

Виконаємо даний експеримент. За початковий обхід беремо граф та початок/ціль на рисунку 6.

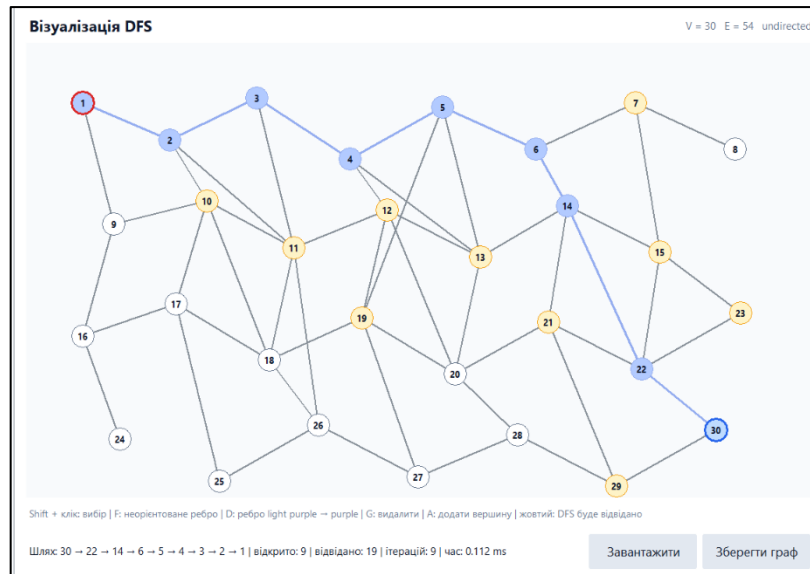


Рис.9 – Обхід неорієнтованого графа змінивши початку та кінцеві вершину місцями.

Для даного графу згідно рисунка 9 краще використовувати шлях від вершини 30 до вершини 1, оскільки задіюється менше пам'яті для пошуку шляху та виконується менше ітерацій (оскільки кількість відкритих вершин 9, відвіданих: 19). При цьому час обходу зменшився: 0.112 с.

Дослідження 3: Видалення та додавання вершин, ребер та дуг, які входять у вирішальний граф.

Проведемо даний дослід та видалимо вершину 7 (див рис.10).

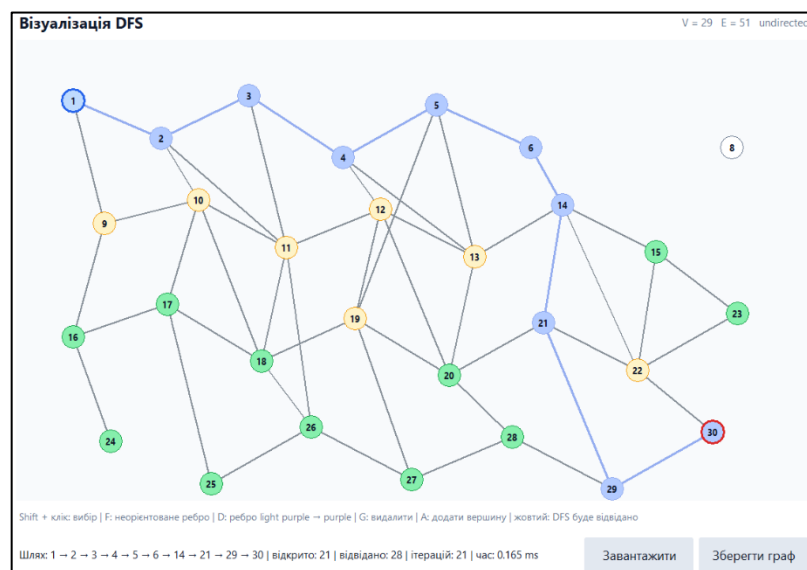


Рис.10 – Обхід неорієнтованого графа видаливши вершину 7.

Результат показав зміну шляху (з 1-2-3-4-5-6-7-15-22-30 на 1-2-3-4-5-6-14-21-29-30), однак тепер граф не відвідав вершину 8, оскільки граф став незв'язним і таким чином зменшилося кількість витраченого часу на пошуку маршруту (відкрито 21 та відвідано 28, хоча початковий обхід мав відкрито 22 та відвідано 30).

Тепер додамо назад вершину 7 яка буде поєднана із вершиною 4 та 22. Також додамо дугу від 13 до 21 (див.рис.11).

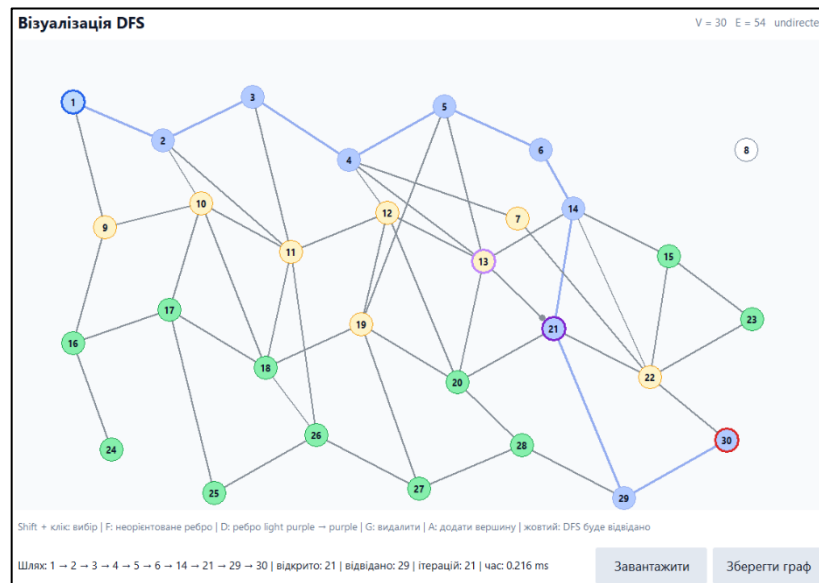


Рис.11 – Обхід неорієнтованого графа додавши вершину 7, ребра та дуги.

На рисунку 11 видно, що шлях не використовує додані ребро та дугу, хоча вони скорочують шлях обходу. Тому змінимо початок і кінець місцями.

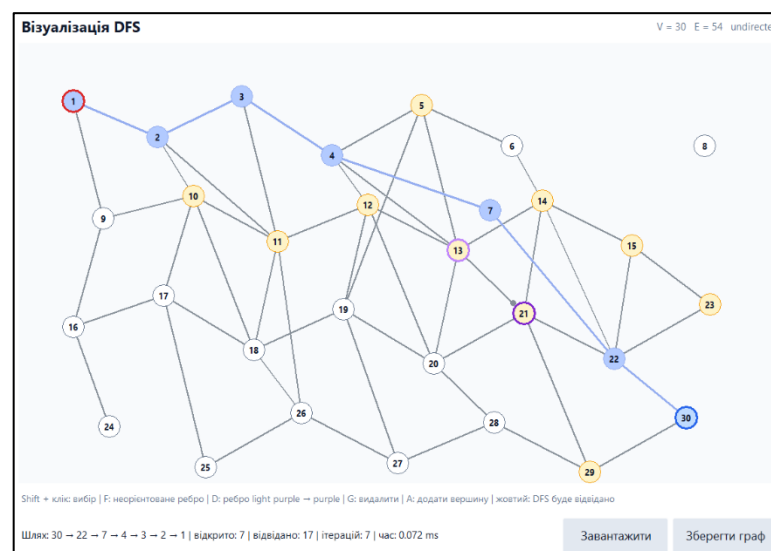


Рис.12 – Обхід неорієнтованого графа змінивши початку та кінцеві вершину місцями.

На рисунку 12 видно, що, змінивши початкову і кінцеву вершину місцями, отримали новий шлях, який на цей раз використовує створені ребра та вершину 7.

При проходженні час виконання програми показував, залежність обходу даного алгоритму від шляху та цілей. Тому можна вважати, що час показує реальний стан виконання алгоритму DFS.

Дослідження 4: Використання пошуку вглиб на різних видах графів.

Завантажимо орієнтований підготовлений граф та запустимо пошук при різних початкових та кінцевих вершинах.

Було вибрано наступні вершини:

Дослід (рис.)	Початкова вершина	Кінцева вершина	Рисунок
1	1	30	13
2	30	1	14
3	14	1	15
4	25	14	16

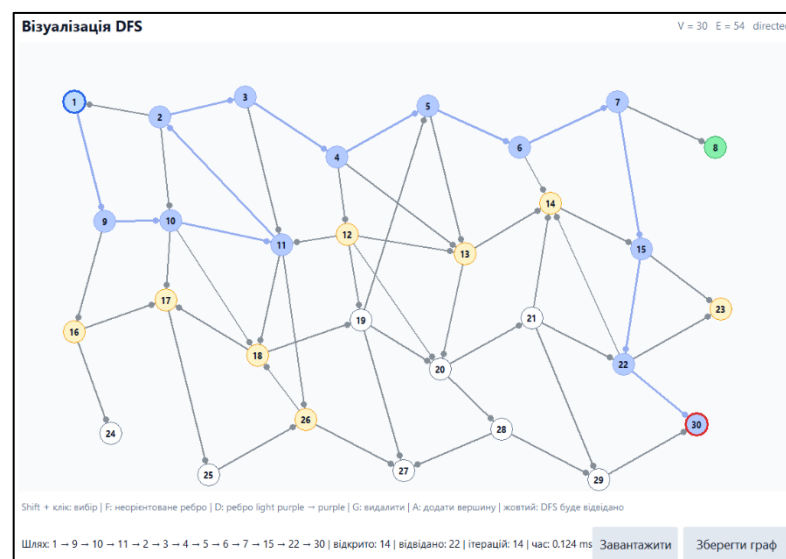


Рис.13 – Обхід орієнтованого графа з початковою вершиною 1 та кінцевою 30.

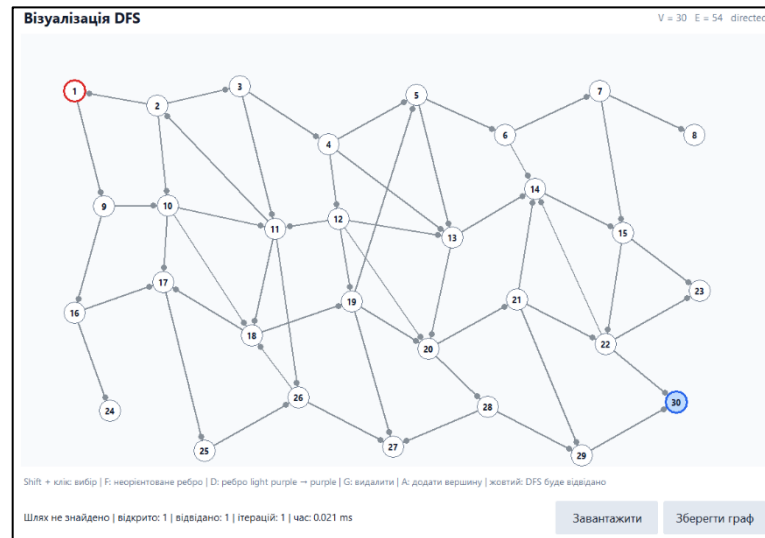


Рис.14 – Обхід орієнтованого графа з початковою вершиною 30 та кінцевою 1.

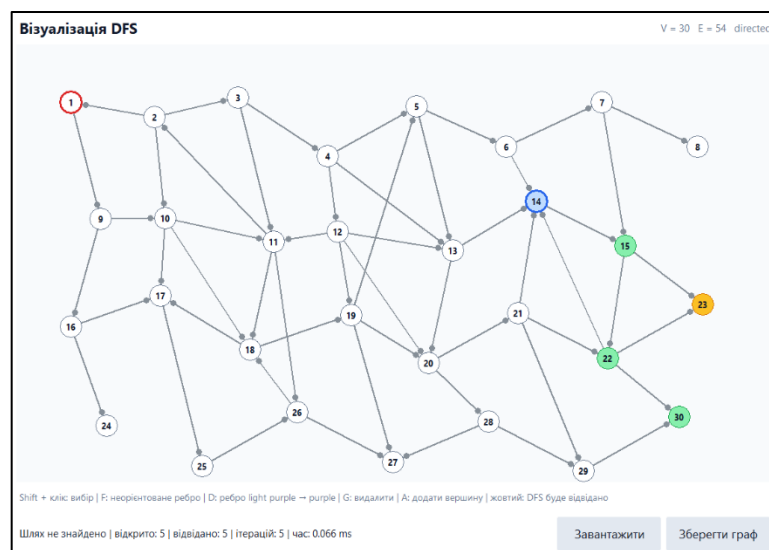


Рис.15 – Обхід орієнтованого графа з початковою вершиною 14 та кінцевою 1.

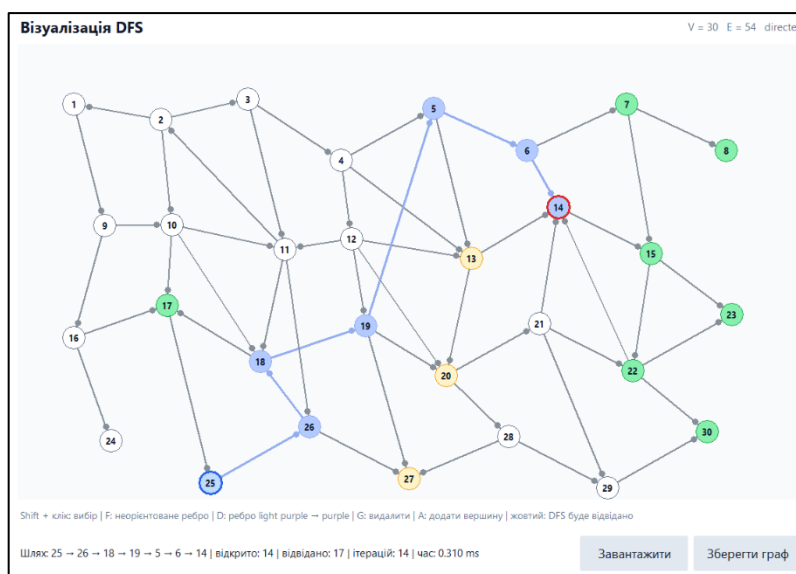


Рис.16 – Обхід орієнтованого графа з початковою вершиною 25 та кінцевою 14.

Як можна проаналізувати досліди 1-4 (з рисунків 13-16), при різних початкових та кінцевих вершин, результуючий шлях можна знайти або його просто не існує. Наприклад: для досліду 1 та 4 були знайдені шляхи, а для 2 та 3 – ні.

Тепер використаємо граф-дерево (див.рис.17) та проведемо декілька експериментів із знаходженням шляху від однієї вершини до різних кінцевих вершин.

Було вибрано наступні вершини:

Дослід (рис.)	Початкова вершина	Кінцева вершина	Рисунок
1	1	15	17
2	1	23	18
3	1	18	19

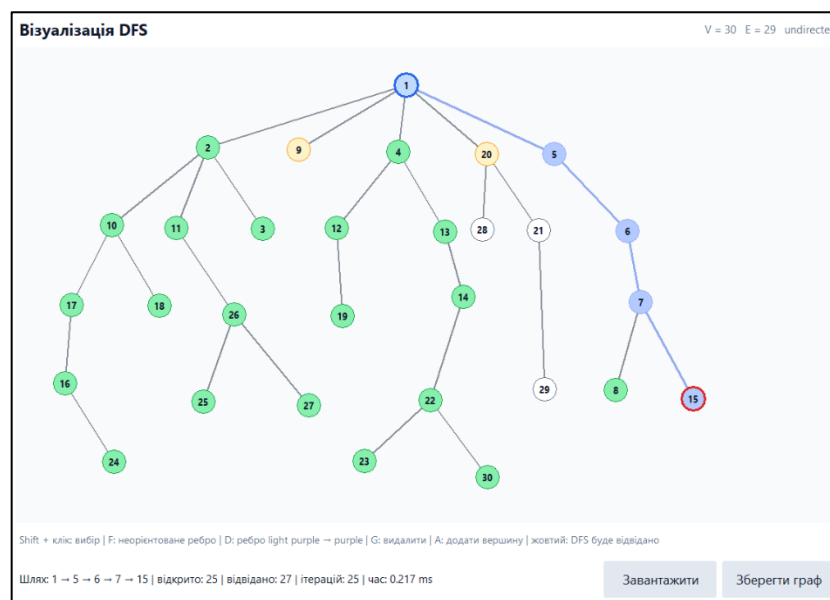


Рис.17 – Обхід дерева з початковою вершиною 1 та кінцевою 15.

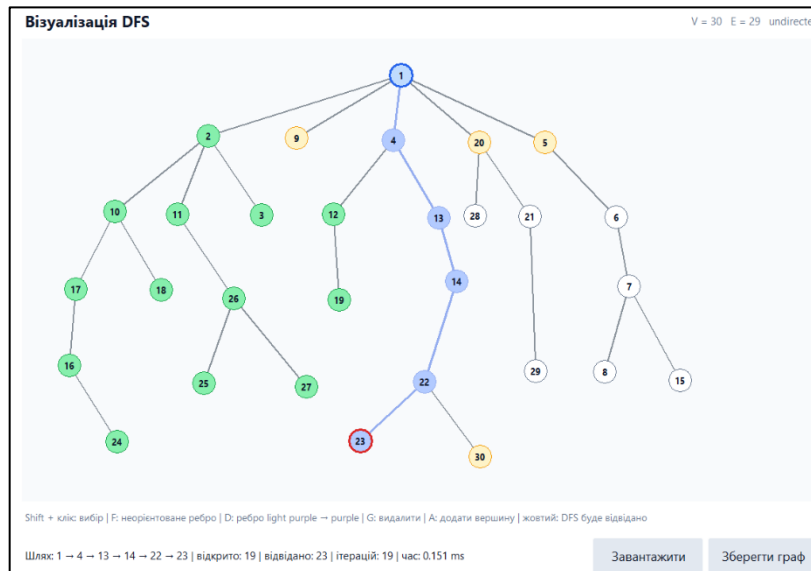


Рис.18 – Обхід дерева з початковою вершиною 1 та кінцевою 23.

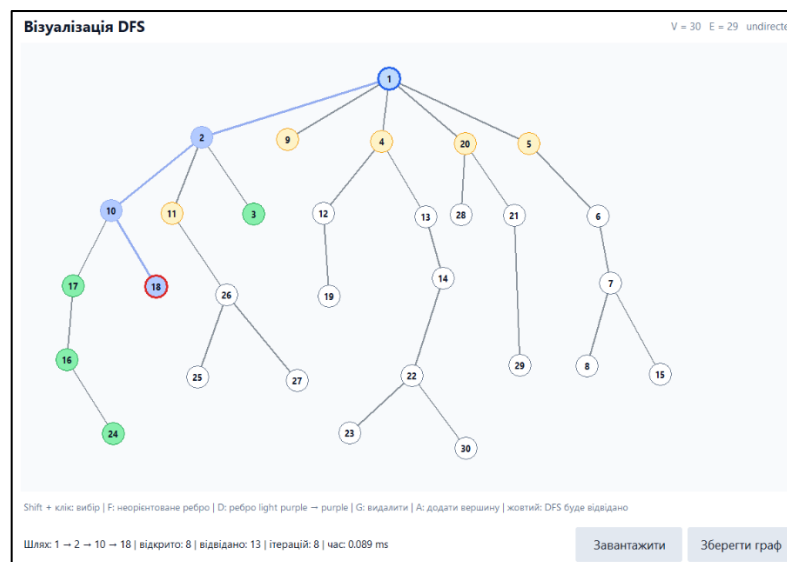


Рис.19 – Обхід дерева з початковою вершиною 1 та кінцевою 18.

З рисунків 17-19 видно, як працює алгоритм пошуку вглиб. Тобто різниця між вибору вершини в певній гілці впливає на кількість відвіданих вершин на усіх кроках даного алгоритму, а також затраченому часу, оскільки в досліді 1 (рис.18) відвідані всі гілки та затрачений час 0.217 с., а в досліді 3 (рис.19) відвідано тільки одну гілку та затрачений час 0.089 с.

Дослідження 5: Порівняння пошуку вшир та пошуку вглиб

Проведемо порівняння роботи алгоритмів пошуку вшир (BFS) та пошуку вглиб (DFS), використовуючи в кожному випадку один і той самий граф та однакові умови пошуку. Для порівняння було використано неорієнтований граф із 30 вершинами та 54 ребрами, який використовувався у попередніх дослідженнях. Початковою вершиною було обрано вершину 1, а кінцевою — вершину 30.

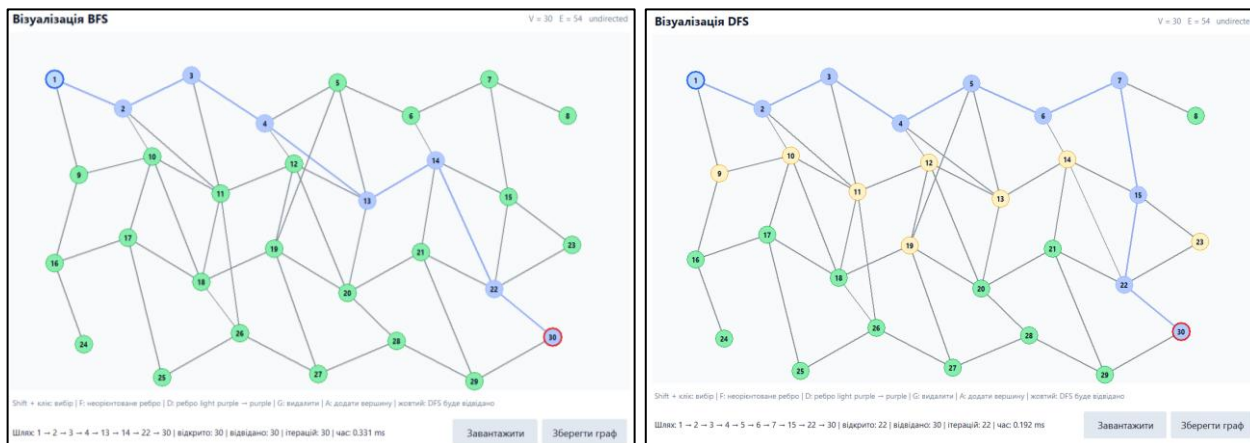


Рис.20 – Порівняння пошуку вшир та пошуку вглиб на одному неорієнтованому графі при початковій вершині 1 та цільовій 30.

При виконанні пошуку вшир алгоритм починає роботу з початкової вершини та послідовно розглядає всі вершини, які знаходяться на однаковій відстані від початкової. Після завершення перевірки вершин одного рівня алгоритм переходить до наступного рівня. Така організація пошуку забезпечує знаходження найкоротшого шляху за кількістю ребер у неваговому графі.

Пошук вглиб працює за іншим принципом. Після вибору наступної вершини алгоритм намагається якомога глибше пройти по відповідній гілці графа, перш ніж повернутися до попередньої вершини та продовжити пошук іншої гілки. Тому порядок обходу вершин при DFS суттєво залежить від порядку вибору суміжних вершин.

Для першого порівняння (рис.20) з початковою вершиною 1 та цільовою 30 отримано такі результати:

Алгоритм	Відкрито вершин	Відвідано вершин	Ітерацій	Час
BFS	30	30	30	0.331 с
DFS	22	30	22	0.192 с

Як видно з результатів, обидва алгоритми можуть знайти шлях між заданими вершинами, однак порядок обходу графа та кількість перевірених вершин відрізняються. Пошук вшир поступово розширює область пошуку навколо початкової вершини, тоді як пошук вглиб спочатку рухається по одній із доступних гілок якомога далі.

Особливо помітною є відмінність у випадку, коли існує декілька можливих шляхів між початковою та кінцевою вершинами. BFS спочатку перевіряє всі можливі шляхи меншої довжини, тому для невагового графа знайдений ним шлях буде найкоротшим за кількістю ребер. DFS не має такої властивості: він

може спочатку пройти довшою гілкою та знайти шлях, який містить більше ребер.

Тепер змінимо початкову та кінцеву вершини місцями, використавши вершину 30 як початкову та вершину 1 як кінцеву.

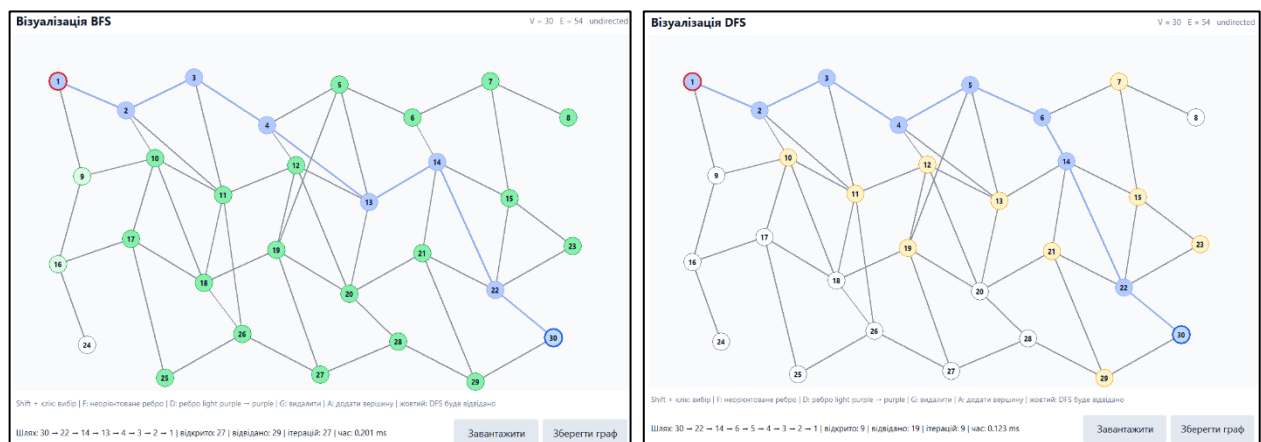


Рис.21 – Порівняння пошуку вшир та пошуку вглиб на одному неорієнтованому графі при початковій вершині 30 та цільовій 1.

Для другого порівняння (рис.21) з початковою вершиною 30 та цільовою 1 отримано такі результати:

Алгоритм	Відкрито вершин	Відвідано вершин	Ітерацій	Час
BFS	27	29	27	0.201 с
DFS	9	19	9	0.123 с

За результатами даного дослідження можна побачити, що зміна напрямку пошуку впливає на обидва алгоритми. Для BFS це визначає рівні, від яких починається поширення пошуку, а для DFS — порядок проходження гілок графа. Таким чином, навіть для одного і того самого графа результати пошуку можуть суттєво змінюватися залежно від початкової вершини.

Проведемо ще одне порівняння на графі-дереві, оскільки структура дерева дозволяє наочно продемонструвати різницю між алгоритмами.

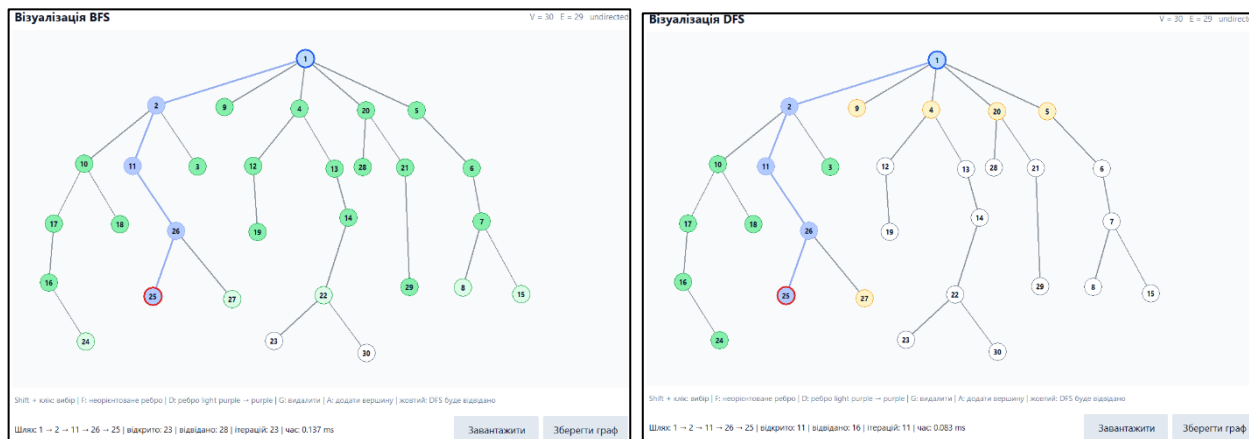


Рис.22 – Порівняння пошуку вшир та пошуку вглиб на одному дереві при початковій вершині 1 та цільовій 25.

Для третього порівняння (рис.22) з початковою вершиною 1 та цільовою 25 отримано такі результати:

Алгоритм	Відкрито вершин	Відвідано вершин	Ітерацій	Час
BFS	23	28	23	0.137 с
DFS	11	16	11	0.083 с

При пошуку на дереві BFS рухається по рівнях. Тобто спочатку перевіряється початкова вершина, після чого всі її нащадки, потім вершини наступного рівня і так далі. Якщо цільова вершина знаходиться на невеликій відстані від початкової, BFS може знайти її, не досліджуючи глибші частини інших гілок.

DFS, навпаки, спочатку обирає одну гілку та продовжує її дослідження до максимально можливої глибини. Якщо цільова вершина знаходиться в цій гілці, шлях може бути знайдений досить швидко. Якщо ж ціль розташована в іншій гілці, DFS спочатку може відвідати значну кількість непотрібних вершин, після чого повернутися назад та продовжити пошук.

Таким чином, основна відмінність між алгоритмами полягає в організації зберігання та вибору наступної вершини для пошуку. BFS використовує принцип черги (*FIFO — First In, First Out*): першою обробляється та вершина, яка була додана до черги першою. DFS використовує принцип стека (*LIFO — Last In, First Out*): першою обробляється остання додана вершина.

Відмінність можна представити наступним чином:

Характеристика	BFS	DFS
Основний принцип	Пошук по рівнях	Пошук по гілках
Структура даних	Черга	Стек
Напрямок пошуку	У ширину	У глибину
Найкоротший шлях у неваговому графі	Гарантовано знаходить	Не гарантується
Залежність від порядку сусідів	Є	Значна
Перевага	Добре працює для близьких цілей	Може швидко знайти глибоку цільову вершину
Недолік	Може зберігати багато вершин одного рівня	Може досліджувати непотрібні гілки

Висновок

У ході виконання лабораторної роботи було розроблено програму реалізації пошуку вглиб (DFS) на графах та досліджено особливості її роботи за різних параметрів пошуку і конфігурацій графа. Розроблений графічний інтерфейс забезпечує візуалізацію графа та процесу пошуку, можливість зміни початкової і цільової вершин, порядку обходу суміжних вершин, редагування структури графа, а також відображення результатів виконання алгоритму.

У процесі експериментів було встановлено, що зміна порядку обходу суміжних вершин безпосередньо впливає на послідовність розкриття вершин, кількість відвіданих вершин та конкретний знайдений маршрут. Для досліджуваного неорієнтованого графа обхід за спаданням дав 11 відкритих та 18 відвіданих вершин, тоді як власний порядок обходу дав 11 відкритих та 20 відвіданих вершин. Це підтвердило залежність DFS від порядку вибору наступної вершини.

Зміна початкової та кінцевої вершин також суттєво вплинула на результати пошуку. При зміні напрямку пошуку з вершини 1 до вершини 30 на пошук від вершини 30 до вершини 1 було отримано іншу послідовність обходу. У другому випадку було відкрито 9 та відвідано 19 вершин, що є меншим показником порівняно з початковим напрямком. Отже, вибір початкової вершини може істотно впливати на кількість опрацьованих вершин та витрати ресурсів.

Під час зміни структури графа шляхом видалення та додавання вершин, ребер та дуг було встановлено, що зміна зв'язків між вершинами може призвести до зміни результуючого маршруту та кількості опрацьованих вершин. Зокрема, після видалення вершини 7 початковий шлях 1-2-3-4-5-6-7-15-22-30 було замінено на шлях 1-2-3-4-5-6-14-21-29-30, а кількість відкритих вершин зменшилася до 21, відвіданих — до 28. Після відновлення вершини та додавання нових зв'язків зміна початкової і кінцевої вершин дозволила отримати маршрут, який використовував створені зв'язки.

Окремо було досліджено роботу DFS на орієнтованому графі та дереві. Для орієнтованого графа було встановлено, що результат пошуку залежить від напрямку дуг та вибраних початкової і кінцевої вершин. У проведених дослідках для частини пар вершин шлях було знайдено, тоді як для інших він був відсутній через напрямленість зв'язків. При дослідженні дерева було продемонстровано характерну особливість DFS: алгоритм спочатку намагається максимально заглибитися в одну з гілок. У випадках, коли цільова вершина знаходиться у вибраній гілці, це дозволяє зменшити кількість відвіданих вершин та час виконання.

Порівняння DFS з BFS на одному графі та за однакових умов пошуку показало відмінності в організації та результатах роботи алгоритмів. Для пошуку

від вершини 1 до вершини 30 BFS відкрив та відвідав 30 вершин і виконав 30 ітерацій за 0.331 с, тоді як DFS відкрив 22 та відвідав 30 вершин, виконав 22 ітерації за 0.192 с. При зміні напрямку пошуку від вершини 30 до вершини 1 BFS відкрив 27 та відвідав 29 вершин за 0.201 с, а DFS відкрив 9 та відвідав 19 вершин за 0.123 с. На дереві при пошуку від вершини 1 до вершини 25 BFS відкрив 23 та відвідав 28 вершин за 0.137 с, тоді як DFS відкрив 11 та відвідав 16 вершин за 0.083 с.

Таким чином, проведене порівняння підтвердило, що BFS і DFS використовують різні принципи організації пошуку. BFS здійснює пошук пошарово, використовуючи чергу, та для невагового графа гарантує знаходження найкоротшого шляху за кількістю ребер. DFS використовує стек і послідовно заглиблюється в окремі гілки графа, тому його результат значною мірою залежить від порядку обходу суміжних вершин і не гарантує найкоротшого маршруту.

Поставлену мету лабораторної роботи досягнуто: реалізовано алгоритм пошуку вглиб, досліджено вплив порядку обходу, зміни початкової та кінцевої вершин, структури та типу графа на результати пошуку, а також проведено безпосереднє порівняння BFS та DFS. Експерименти показали, що вибір алгоритму залежить від поставленої задачі та структури графа: BFS є доцільним для пошуку найкоротшого шляху в неваговому графі, тоді як DFS може бути ефективнішим за кількістю опрацьованих вершин і часом у випадках, коли цільова вершина знаходиться в гілці, яку алгоритм обирає першою.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії https://github.com/MrCatman137/AI_systemem у директорії lab_2 та має повну інструкцію запуску даної програми.